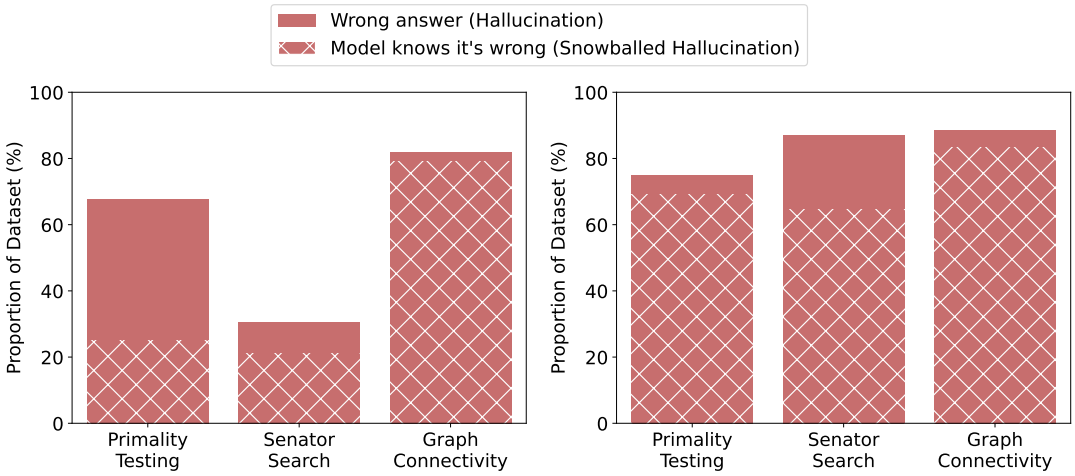


(a) ChatGPT (b) GPT-4

Figure 2: Percentage of hallucination and percentage of snowballed hallucination (both calculated with respect to the entire dataset) for ChatGPT and GPT-4. The precise numbers for this plot are available in Table 6 and Table 7 in the Appendix.

3.2 Inference Setup

Language models. We run all experiments on ChatGPT (`gpt-3.5-turbo`) and GPT-4 with greedy decoding.

Our experiments are *zero-shot* (i.e., we do not show the model any example QA pairs in the prompt). We focus on the model behavior under the direct prompt (see §A for full examples), which is the most common way users interact with LMs. See §4 for experiments with the zero-shot chain-of-thought style prompting method.

For each dataset, we perform a two-stage evaluation. First, we evaluate the model's accuracy (i.e., how many of the questions it answers correctly). When either models is *incorrect*, empirically it *always* generates a justification. In the second stage, we assess whether the model can identify the incorrect step in the explanation.

For a given question, we evaluate the model's response by examining whether the output begins with either `Yes` or `No`. In cases where the response does not fall into these categories, we manually determine the answer conveyed by the model.

3.3 LM Recognition of Snowballed Hallucinations

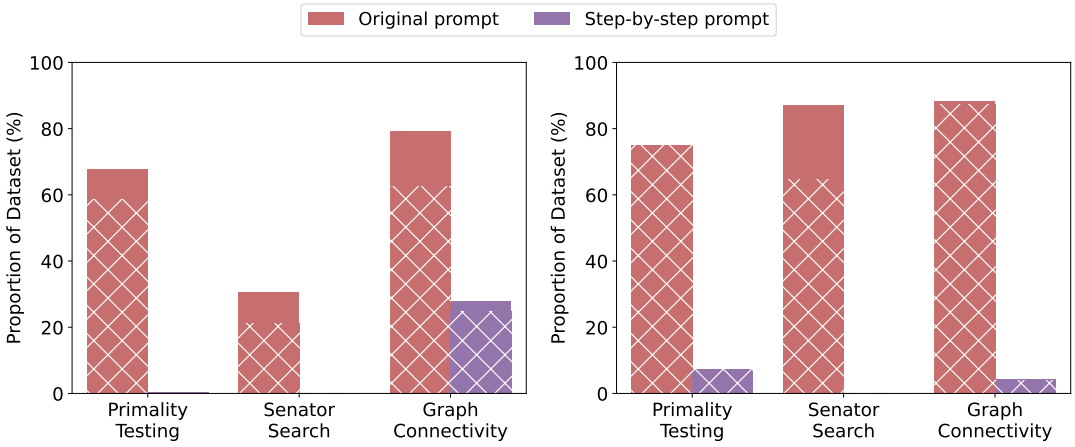
We probe whether LMs recognize their snowballed hallucinations by verifying the model's incorrect claims in the output against the model itself. Note that our recognition procedure relies on heuristics gained from manual examination of the model output, and these heuristics might not work on other models (e.g., a different model might not provide factors when supporting the claim that a number is not prime).

Graph Connectivity For each sample where the model thinks there is a series of connecting flights (where answer starts with `Yes`), we manually extract the list of flights from the model's output and identify the invalid or discontinuous flights.

We then, in a new session, ask the model to verify whether the extracted flights are valid based on the flight information, and if consecutive flights are indeed connected. We manually assess the verification output to check if the model correctly detects the error. See Appendix Table 3 for how we prompt the model and an example of successful verification.

Primality Testing For each sample where the model answers that the number is not prime, we extract the factors the model uses to justify it. The extraction is done by putting the output in the context and asking *"What are the factors proposed in the above text? List them out."* We use ChatGPT for extraction with one-shot demonstration (for its fast inference speed); we manually checked 30 examples and found that it can always extract the correct factors.

We then, in a new session, ask the model to verify each extracted factor individually. See Appendix Table 4 for an example of successful verification.



(a) ChatGPT (b) GPT-4

Figure 3: Error rate and snowballed hallucination rate (hatch pattern) for ChatGPT and GPT-4, when using the original prompt versus *"Let's think step by step"*. See Appendix Table 8 and Table 9 for the exact numbers.

Senator Search For each sample where the model thinks there is such senator, we extract the name of the senator the model uses to justify the existence, by putting the output in the context and asking *"What is the senator mentioned in the above text? Just give the name"*. Again, we use ChatGPT and manually observed perfect extraction on 30 examples.

We then, in a new session, ask the model if that senator's alma mater is the college in the question and has represented the state in the question. See Appendix Table 5 for an example of successful detection.

3.4 Results

Question-answering accuracy Figure 2 shows that both ChatGPT and GPT-4 experience very low accuracy across the board. With the exception of ChatGPT on the **Senator Search** dataset, all models achieve less than 50% accuracy.(See Appendix Table 6 for a breakdown of the error rate by dataset.) We observe that GPT-4 performs worse than ChatGPT across all datasets despite popularly being considered superior to ChatGPT (OpenAI, 2023). While ChatGPT has an average accuracy of 39.87%, GPT-4 has only 16.6%.

Hallucination detection Here, we check whether the model can identify that the incorrect claim is wrong when it is presented alone. As shown in Figure 2, ChatGPT detects 67.37% of incorrect claims in explanations (i.e., snowballed hallucinations), and GPT-4 detects 87.03%. Notice that when the model fails the verification (an example in Appendix Table 12), we do not consider it a snowballed hallucination.

Overall, we find that ChatGPT and GPT-4 are both extremely susceptible to hallucination snowballing, leading to extremely simple mistakes.

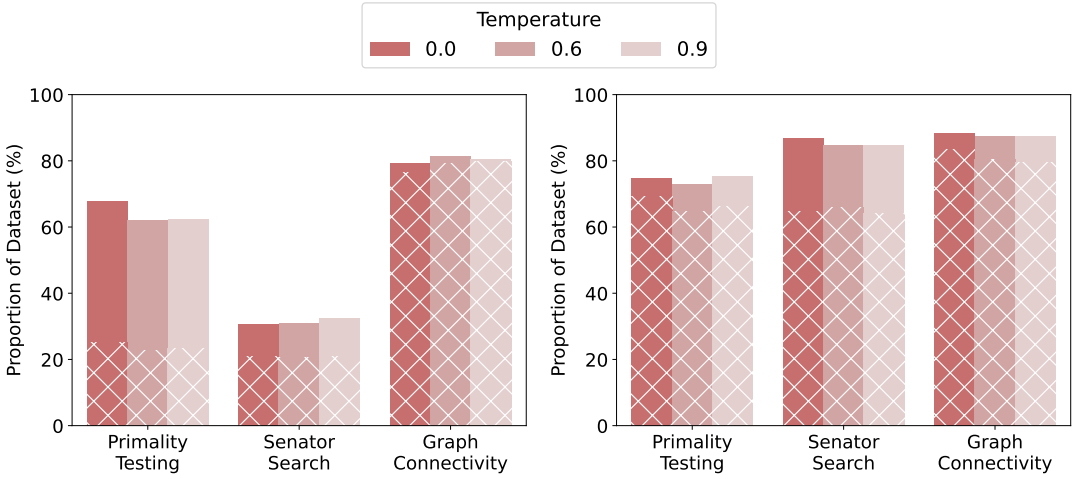
4 Can we prevent snowball hallucinations?

We hypothesize that hallucination snowballing occurs because LMs are trained to model continuations consistent with their current context (the given prompt and prior outputs). Although a fix to the fundamental problem might require more than just inference-time modification, in this section we study the effectiveness of two inference strategies in alleviating hallucination snowballing: prompting (§4.1) and decoding or training methods (§4.2).

4.1 Engineering Better Prompts

In this section, we examine the effectiveness of better prompts on preventing snowballed hallucination by using a different zero-shot prompt that encourages the model to generate the reasoning chain before the answer. Since the outputs generated under these prompts are less structured, we manually inspect them to determine correctness and the presence of snowballed hallucinations.

For each task, we append *"Let's think step-by-step"* at the end of the original question (shown in Table 1). As shown in Figure 3, the model can solve the **Senator Search** task perfectly, achieve $\leq$10% error rate on **Primality Testing**, and $\leq$30% on **Graph Connectivity**. Despite the large improve-



(a) ChatGPT

(b) GPT-4

Figure 4: Error rate and snowballed hallucination rate (hatch pattern) from ChatGPT and GPT-4, when using different values for temperature at decoding-time. See Appendix Table 10 and Table 11 for the exact numbers.

ment in accuracy, we identify a potential issue: the model sometimes hallucinate while outputting the reasoning chain, which causes snowballed hallucination in future steps. For example, in the below output,

> *[....previous steps omitted]*
>
> *Step 3: From city E, we have three options: a flight to city N, a flight to city B, or a flight to city C.*
>
> *Step 4: The only option that could potentially lead us to city M is the flight from city E to city C.*
>
> *[....rest of the output omitted]*

ChatGPT incorrectly states that there are three options in the step 3 (there are only two), inducing the snowballed hallucination *"or a flight to city C"* (ChatGPT can verify that E → C is not a valid flight in a separate session). As shown in Figure 3, GPT-4 still has a high overall snowballed hallucination rate at 94.90% averaged across tasks, and ChatGPT also obtains a similarly high snowballed hallucination rate.

Finally, while our experiments have focused on simple multi-step problems that are suitable for breaking down step-by-step, we hypothesize that hallucination snowballing appears in open-ended text generation more broadly, where one mistake in the generation triggers more (Arora et al., 2022). In these cases, better prompting would neither be able to anticipate nor fix these mistakes.

4.2 Algorithmic Corrections

Increasing the temperature During decoding, the temperature t controls the sharpness of the output distribution, with higher t spreading probability mass away from the model's most likely prediction for each next word. Our experiments in §3 used greedy decoding, which is equivalent to $t = 0$. At $t = 0.6$ and $t = 0.9$, both error rates and snowballed hallucination rate remain similarly high, in both GPT-4 and ChatGPT (Figure 4).

Top-k and nucleus sampling Using sampling methods such as top-k sampling or nucleus sampling (Holtzman et al., 2020) would not help since they only *narrow* the range of tokens to be considered, and thus can only *increase* the probability that the model will immediately commit to an answer.

Beam search The argument for hallucination snowballs in §2 relies on the fact that, once a model generates some tokens committing to an answer, they remain in the context and influence later generations. One potential way around this is *beam search*, i.e., maintaining a beam of high-probability sequences at each timestep rather than a single sequence. In principle, if some sequences in the beam after the initial token do not commit to an answer (or commit to the right answer), their continuations may eventually have higher probability than those that initially commit incorrectly and later produce incorrect reasoning as a result. If so, beam search would solve the snowball hallucination problem. Unfortunately, we cannot test the effect of beam search on hallucination snowballs because the Ope-